Documentation – Projet Auchan

1. Présentation du projet

Le Centre Commercial AUCHAN souhaite une nouvelle plateforme e-commerce. Mon rôle : réaliser une application web dynamique, modulaire et responsive, côté client uniquement, avec **HTML5, Tailwind CSS, JavaScript Vanilla ES6+ et Git/GitHub**.

L'application permet de consulter un catalogue de 12 produits, de rechercher, filtrer par catégorie, paginer, puis (à venir) d'ajouter au panier par glisser-déposer.

2. Structure du projet

```
auchan/
├── index.html
├── data/data.json        (catalogue initial)
├── css/                  (input.css et output.css Tailwind)
├── assets/pictures/      (logo et images produits)
└── js/
    ├── main.js           (point d'entrée, état, événements)
    ├── api.js            (fetch)
    ├── storage.js        (localStorage)
    ├── products.js       (logique : recherche, filtre, pagination)
    ├── ui.js             (DOM : affichage)
    ├── cart.js           (panier et tarification)
    ├── dragDrop.js       (glisser-déposer)
    └── validation.js     (formulaire Contact, Regex)
```

Règle suivie : **chaque fichier a une seule responsabilité**. La logique (`products.js`) ne touche jamais au DOM, le DOM est dans `ui.js`, et les événements sont dans `main.js`.

3. Concepts utilisés

3.1 Modules ES (import / export)

Chaque fichier est un module isolé. `export` rend une fonction utilisable ailleurs, `import` la récupère. Le script est chargé avec `type="module"`, ce qui impose un serveur local (Live Server).

```
// api.js
export async function fetchproduct() { ... }

// main.js
import { fetchproduct } from "./api.js";
```

L'extension `.js` est obligatoire dans le chemin, sinon le navigateur renvoie une erreur 404.

3.2 Fetch, async / await et gestion d'erreur

`fetch` envoie une requête et renvoie une **Promise**. `await` attend la réponse. `fetch` ne lève pas d'erreur sur un 404 ou un 500 : il faut tester `response.ok` et utiliser `throw`.

```
export async function fetchproduct() {
  const response = await fetch("./data/data.json");
  if (!response.ok) throw new Error(`Erreur ${response.status}`);
  return response.json();
}
```

L'appel est protégé par `try / catch` dans `main.js`. Les codes HTTP : 2xx succès, 3xx redirection, 4xx erreur du client, 5xx erreur du serveur.

3.3 JSON et localStorage

`localStorage` ne stocke que du texte. `JSON.stringify` transforme un tableau en texte (sérialisation), `JSON.parse` fait l'inverse (désérialisation).

```
const PRODUCTS_KEY = "products";

export function saveproduct(products) {
  localStorage.setItem(PRODUCTS_KEY, JSON.stringify(products));
}
export function leadproduct() {
  const data = localStorage.getItem(PRODUCTS_KEY);
  return data ? JSON.parse(data) : null;
}
```

Au premier chargement, `localStorage` est vide : on fait un `fetch` puis on sauvegarde. Ensuite on lit directement dans `localStorage`, donc les données restent après un rafraîchissement. Quand `data.json` change, il faut vider l'ancienne clé (`localStorage.removeItem("products")`).

3.4 ES6+ : arrow functions, destructuring, template literals

```
const createproductCart = ({ id, name, category, price, image }) => `
  <article data-id="${id}">
    <img src="${image}" alt="${name}">
    <h2>${name}</h2>
    <p>${price} €</p>
  </article>`;
```

- **Arrow function** : syntaxe courte `() => ...`.
- **Destructuring** : `({ name, price })` extrait directement les propriétés de l'objet.
- **Template literal** : les backticks et `${...}` insèrent des valeurs dans du HTML.

L'affichage se fait avec `map()` puis `join("")` :

```
container.innerHTML = products.map(createproductCart).join("");
```

3.5 filter() : recherche et catégorie

`filter()` parcourt un tableau et renvoie **un nouveau tableau** avec les éléments qui respectent la condition. Le tableau d'origine ne change pas.

```
export const searchproduct = (products, query) =>
  products.filter(({ name }) =>
    name.toLowerCase().includes(query.toLowerCase().trim()));

export const filterByCategory = (products, category) =>
  category === "all"
```

```
    ? products
    : products.filter((product) => product.category === category);
```
`toLowerCase()` rend la recherche insensible à la casse, `includes()` teste si le nom contient le texte saisi.

3.6 L'état (state) et la fonction update()

Un objet `state` mémorise ce que l'utilisateur a choisi. Chaque événement modifie `state`, puis `update()` filtre, découpe et affiche.

```
let products = [];
const state = { query: "", category: "all", page: 1, perPage: 4 };

const update = () => {
  const searched = searchproduct(products, state.query);
  const filtered = filterByCategory(searched, state.category);
  const totalPages = getTotalPages(filtered, state.perPage);
  renderProduct(paginate(filtered, state.page, state.perPage));
  renderPagination(totalPages, state.page);
};
```
Grâce à `state`, la recherche et la catégorie fonctionnent **ensemble**. On filtre toujours à partir du tableau complet, donc effacer la recherche ramène tous les produits.

3.7 Pagination

`slice(début, fin)` découpe le tableau, et `Math.ceil` calcule le nombre de pages.

```
export const paginate = (products, page, perPage) => {
  const start = (page - 1) * perPage;
  return products.slice(start, start + perPage);
};

export const getTotalPages = (products, perPage) =>
  Math.max(1, Math.ceil(products.length / perPage));
```
Exemple : 12 produits, 4 par page → `Math.ceil(12 / 4) = 3` pages. Page 2 → `slice(4, 8)`.

Les boutons de pages sont recréés à chaque affichage, donc on utilise la **délégation d'événement** : un seul `click` sur `#pagination`, et `closest("button[data-page]")` retrouve le bouton cliqué. L'attribut `data-page` stocke le numéro de la page, lu avec `button.dataset.page`. Un menu permet de choisir 4, 8 ou 12 produits par page.

3.8 Code synchrone

Un code synchrone s'exécute **ligne par ligne**. Chaque instruction attend que la précédente soit terminée.

```
console.log("1");
console.log("2");
console.log("3");
// Affiche : 1, 2, 3
```
Simple et prévisible, mais si une instruction est longue, tout le reste attend et la page se fige.

3.9 Code asynchrone

Un code asynchrone lance une tâche longue puis **continue sans l'attendre**. Le résultat arrive plus tard, et on indique ce qu'il faut faire à ce moment-là. Exemples :

- `fetch` : requête vers un serveur ou un fichier.
- `setTimeout` / `setInterval` : minuteurs.
- Les événements (`click`, `input`, `change`).

3.10 Pourquoi JavaScript en a besoin

JavaScript n'a qu'**un seul fil d'exécution** (single thread) : il ne fait qu'une chose à la fois. Si on attendait la réponse du serveur de façon synchrone, la page serait bloquée (plus de clic, plus de défilement). L'asynchrone laisse le navigateur travailler pendant l'attente.

3.11 Exemple avec setTimeout

```
console.log("1");
setTimeout(() => console.log("2"), 0);
console.log("3");
// Affiche : 1, 3, 2
```

Même avec 0 ms, « 2 » arrive en dernier : `setTimeout` est mis de côté et n'est exécuté qu'après la fin du code synchrone.

3.12 La boucle d'événements (event loop)

Les tâches asynchrones terminées attendent dans une file. Quand la pile d'appels est vide, la **boucle d'événements** prend la tâche suivante et l'exécute. C'est pourquoi le code synchrone passe toujours en premier. Les `.then()` des Promises sont traités avant les `setTimeout`.

3.13 Les callbacks

Un **callback** est une fonction donnée à une autre fonction, qui l'appelle quand la tâche est finie.

```
setTimeout(() => {
  console.log("Terminé");
}, 1000);
```

Problème : des callbacks imbriqués les uns dans les autres rendent le code illisible (« callback hell »). Les Promises règlent ce problème.

3.14 Les Promises

Une **Promise** représente une valeur qui arrivera plus tard. Elle a trois états :

- **pending** : en attente.
- **fulfilled** : réussie, avec un résultat.
- **rejected** : échouée, avec une erreur.

```
fetch("./data/data.json")
  .then((response) => response.json())
  .then((products) => console.log(products))
  .catch((error) => console.error(error));
```

`.then()` reçoit le résultat en cas de succès, `.catch()` reçoit l'erreur.

3.15 async / await

`async` / `await` est une écriture plus lisible des Promises : le code asynchrone ressemble à du code synchrone. `await` met en pause **la fonction**, pas toute la page, et ne s'utilise que dans une fonction `async`. Une fonction `async` renvoie toujours une Promise.

```
const loadProducts = async () => {
  const response = await fetch("./data/data.json");
  return response.json();
};
```

3.16 try / catch / finally

```
try {
  const products = await loadProducts();
} catch (error) {
  console.error("Erreur :", error.message);
} finally {
  console.log("Toujours exécuté");
}
```

`try` tente le code, `catch` attrape l'erreur, `finally` s'exécute dans tous les cas (par exemple pour cacher un indicateur de chargement).

3.17 Plusieurs tâches en parallèle : Promise.all

Attendre les tâches une par une est plus lent. `Promise.all` les lance ensemble et attend qu'elles soient toutes terminées. Si une seule échoue, le tout est rejeté.

```
const [products, users] = await Promise.all([
  fetch("./data/data.json").then((r) => r.json()),
  fetch("https://randomuser.me/api/").then((r) => r.json()),
]);
```

3.18 Dans mon projet

Dans `main.js`, seul le chargement des données est asynchrone. Le reste (sauvegarde, affichage) est synchrone.

```
const init = async () => {
  try {
    products = await fetchproduct();  // asynchrone : on attend
    saveproduct(products);            // synchrone
    update();                         // synchrone : affichage
  } catch (error) {
    console.error(error);
  }
};
init();
```

3.19 À retenir

- **Synchrone** : une instruction après l'autre, chacune bloque la suivante.
- **Asynchrone** : on lance la tâche et on continue, le résultat arrive plus tard.

- **Callback → Promise → async / await** : trois façons de gérer l'asynchrone, de la plus ancienne à la plus lisible.
- `fetch` ne rejette pas sur une erreur 404 ou 500 : tester `response.ok`.
- Toujours protéger un `await` avec `try / catch`.

3.20 Tailwind CSS et Mobile-First

On écrit d'abord le style du téléphone **sans préfixe**, puis on ajoute des préfixes pour les écrans plus grands : `sm:` (640 px), `md:` (768 px), `lg:` (1024 px). Un préfixe s'applique à partir de cette largeur.

```
<div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
```

Résultat : 1 colonne sur téléphone, 2 sur petite tablette, 4 sur ordinateur. Chaque classe Tailwind applique un seul style, d'où plusieurs classes avec `md:` quand on change plusieurs choses.